23I501 DESIGN AND ANALYSIS OF ALGORITHMS · BATCH 16

Stock Trading: Best Trading Window in Distributed Tick Data

Divide and Conquer with a composable four-field summary

Problem. A brokerage stores net price changes for one stock across **8 servers**, each holding a contiguous date range (25 years of minute ticks, about **2.4×10^6** values). Find the contiguous window with the **maximum total gain**.

Sankar S V · PSG College of Technology · Odd Semester 2026–27

PROBLEM MODELLING

Input, output, objective and scale

Given instance (₹ per day)

+3	−5	+8	−2	+6	−9	+4	+7	−3	−6	+5	+2
1	2	3	4	5	6	7	8	9	10	11	12

Objective: choose $1 \le i \le j \le n$ to maximise

$$G(i, j) = A[i] + A[i+1] + \ldots + A[j]$$

Output: the best sum and its window. Answer: **₹14 on days 3–8**.

Scale: $n \approx 2.4 \times 10^6$, split into 8 contiguous blocks of about 300,000 values.

Decision	Choice
Window	Non-empty (all-negative data → best single tick)
Empty input	Rejected with an error
Ties	Max sum → earliest start → earliest end
Indexing	1-based in report and output, 0-based inside the code
Distributed need	Each server's result must be **composable** without shipping its ticks

DESIGN TECHNIQUE

Why Divide and Conquer?

Cut the range anywhere. Every window of **L · R** is exactly one of three kinds:

Left
entirely in L

Right
entirely in R

Crossing
suffix of L + prefix of R

Each half can be described by a **constant-size summary**, and two adjacent summaries merge in **O(1)**.

The merge is **associative**, so any balanced merge tree works, provided left-to-right order is kept.

Divide · Conquer · Combine

- **Divide:** split by index (midpoint, or server boundary)
- **Conquer:** same problem on each contiguous block
- **Combine:** 4-field merge, O(1)

The same summary serves **8 servers**, a **segment tree** and a **disjoint sparse table**.

We do not claim D&C beats Kadane's O(n) sequentially. The gain is composability.

CORE IDEA

Four-field summary and the merge

$S = (\text{total}, \text{prefix}, \text{suffix}, \text{best})$

Field	Meaning (non-empty)
total	sum of the whole segment
prefix	best sum starting at the left edge
suffix	best sum ending at the right edge
best	best window anywhere

Leaf: $S(x) = (x, x, x, x)$

Plus metadata kept separate: prefix_end, suffix_start, best_start, best_end (for reconstructing windows).

$$\text{total} = \text{L.total} + \text{R.total}$$

$$\text{prefix} = \max(\text{L.prefix}, \text{L.total} + \text{R.prefix})$$

$$\text{suffix} = \max(\text{R.suffix}, \text{R.total} + \text{L.suffix})$$

$$\text{best} = \max(\text{L.best}, \text{R.best}, \mathbf{\text{L.suffix} + \text{R.prefix}})$$

The **bold** term is the best **crossing** window.

Sequential recursion: $T(n) = 2T(n/2) + \Theta(1) = \Theta(n)$, with O(log n) stack.

Q1 · WORKED TRACE (GENERATED BY THE CODE)

Merging the given instance, level by level

Level	Summaries (total, prefix, suffix, best) · best window
0 · leaves	S(x) = (x, x, x, x) for 3, −5, 8, −2, 6, −9, 4, 7, −3, −6, 5, 2
1 · pairs	[1,2] (−2, 3, −2, 3) · [3,4] (6, 8, 6, 8) · [5,6] (−3, 6, −3, 6) · [7,8] (11, 11, 11, 11) · [9,10] (−9, −3, −6, −3) · [11,12] (7, 7, 7, 7)
2 · fours	[1,4] (4, 6, 6, 8) best [3,3] · [5,8] (8, 8, 11, 11) best [7,8] · [9,12] (−2, −2, 7, 7) best [11,12]
3 · eight	[1,8] = merge([1,4], [5,8]) → **(12, 12, 14, 14)** · [9,12] carried unchanged
4 · root	**[1,12] = merge([1,8], [9,12]) → (10, 12, 12, 14) · best window [3, 8]**

Key merge: [1,4] + [5,8]

$$\text{best} = \max(8, 11, 6 + 8 = 14) = 14$$

$$\text{prefix} = \max(6, 4 + 8 = 12) = 12$$

$$\text{suffix} = \max(11, 8 + 6 = 14) = 14$$

The suffix of [1,4] is A[3..4] = 6, and the prefix of [5,8] is the whole block, 8. Together they cross the boundary.

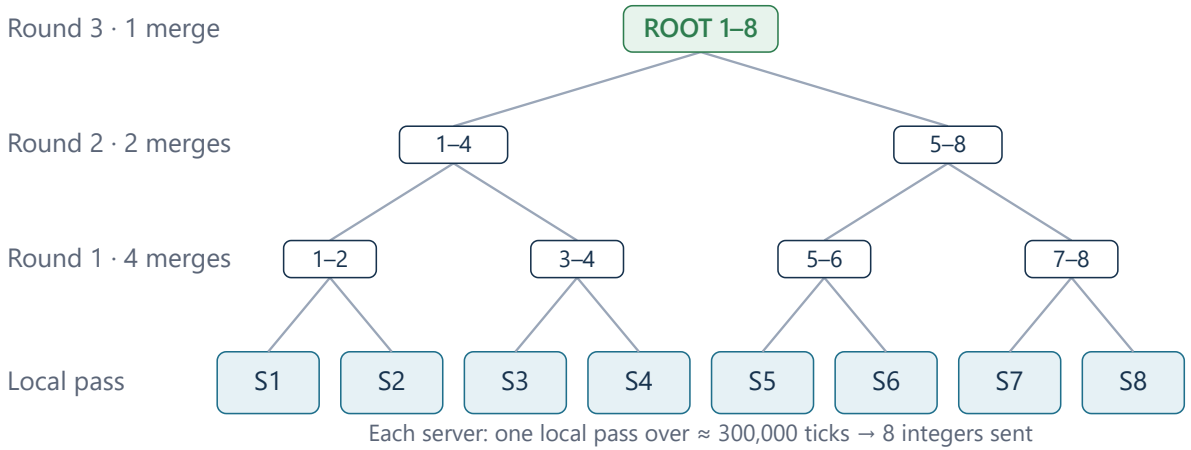
Answer

₹14 on days **3** → **8**

$$8 - 2 + 6 - 9 + 4 + 7 = 14$$

At the root, the crossing candidate is 14 + (−2) = 12 and the right block's best is 7, so the left 14 survives. The program output matches.

Summaries travel, ticks stay



Quantity	Cost (p servers)
Local work per server	$\Theta(n/p)$
Total work	$\Theta(n) + \Theta(p) = \Theta(n)$
Coordinator merges	$p - 1 = 7$, each $O(1)$
Merge depth	$\lceil \log_2 p \rceil = 3$ rounds
Communication	p records of O(1) size, i.e. 64 integers
Ideal parallel time	**$O(n/p + \log p)$**

Assumes balanced blocks, one core per server, data already local, and network or synchronisation cost negligible (or modelled separately). On one machine our simulation shows correctness, **not** speedup. No cluster timings are claimed.

Divide and Conquer vs Kadane's O(n) scan

Aspect	Kadane scan	D&C summary
Time	Θ(n)	Θ(n) total work
Extra space	O(1)	O(log n) stack
Constant factor	smaller	2n − 1 calls, 8-field records
Parallelism	sequential (E[i] needs E[i−1])	independent blocks, O(n/p + log p)
Composability	no	associative O(1) merge
Range queries	rescan [L, R]	segment tree node value
Point updates	rescan	O(log n) with a segment tree

Kadane wins when…

one machine, one full scan or a few queries: same Θ(n), less memory, smaller constants.

D&C wins when…

data is already partitioned, or many range queries and updates reuse the summary.

Recursion depth for $n = 2.4 \times 10^6$ is 23 frames. The $2n - 1$ calls happen over time, not on the stack at once. Network latency per round can erase the benefit of extra servers, so there is no automatic 8× speedup.

Q4 · REPEATED "BEST WINDOW BETWEEN DAY L AND DAY R"

Three structures, one associative summary

Structure	Build	Query	Point update	Space	At $n \approx 2.4 \times 10^6$
Direct scan (Kadane on [L, R])	O(1)	O(R − L + 1)	O(1)	O(1)	Fine for a few queries, but each long query costs millions of steps
Segment tree of summaries	**O(n)**	**O(log n), ≈ 22 levels**	**O(log n)**	**O(n)**	**Recommended:** frequent queries plus occasional corrections
Disjoint sparse table	O(n log n)	O(1) merges	rebuild	O(n log n)	Only for frozen history with huge query volume

Order matters

The merge is associative but **not commutative**. Queries must combine pieces left to right.

Why not a classic sparse table?

The overlapping trick relies on min(x, x) = x. Our merge is **not idempotent**, so overlap would double-count ticks. The disjoint variant uses one ordered merge of two disjoint pieces.

All three are implemented and tested against direct scans on every (L, R) pair of random arrays. Memory is discussed asymptotically and was not measured.

TECHNIQUE SELECTION

Why not the other four techniques?

Technique	Concrete evidence	Verdict
Greedy	Rule "never bridge a negative": on [5, −1, −1, −1, 5] it returns 5, but the optimum is the whole array, 7. Refutes that rule, not every greedy idea.	rule is wrong
Dynamic Programming	Kadane, E[i] = max(A[i], E[i−1] + A[i]), is a **correct** O(n) DP. Its state depends on everything to the left, so it is not a self-contained segment value for servers or tree nodes.	correct not the primary architecture
Backtracking	Search space n(n+1)/2. At n = 2.4 × 10^6 that is **2,880,001,200,000** windows, about 6 × 10^5 times the ≈ 4.8 × 10^6 steps of a linear method.	$\Theta(n^2)$ benchmarked baseline
Branch & Bound	Pruning needs a sound and tight bound. With signed data, "current + all remaining positives" is far too optimistic, so little is pruned and the search stays $\Theta(n^2)$ in the worst case. Tight bounds are the prefix and suffix summaries themselves.	no effective bound

Q5 · CHANGED REQUIREMENT

At most two non-overlapping windows

The 4-field summary stores **one** window per segment, so it cannot represent pairs. Extending the state would work, but a prefix/suffix DP is simpler to prove.

$$P[i] = \text{best window in } A[1..i] \quad \text{(forward Kadane)}$$
$$Q[i] = \text{best window in } A[i..n] \quad \text{(backward Kadane)}$$
$$\text{answer} = \max(\, P[n],\ \max_{i<n} P[i] + Q[i+1] \,)$$

Every pair [a,b], [c,d] with b < c is counted at split i = b. A single window covers the "at most" case. For n = 1 the answer is A[1].

Time O(n), space O(n). For up to k windows, a DP runs in O(kn) time with O(k) state.

Given instance

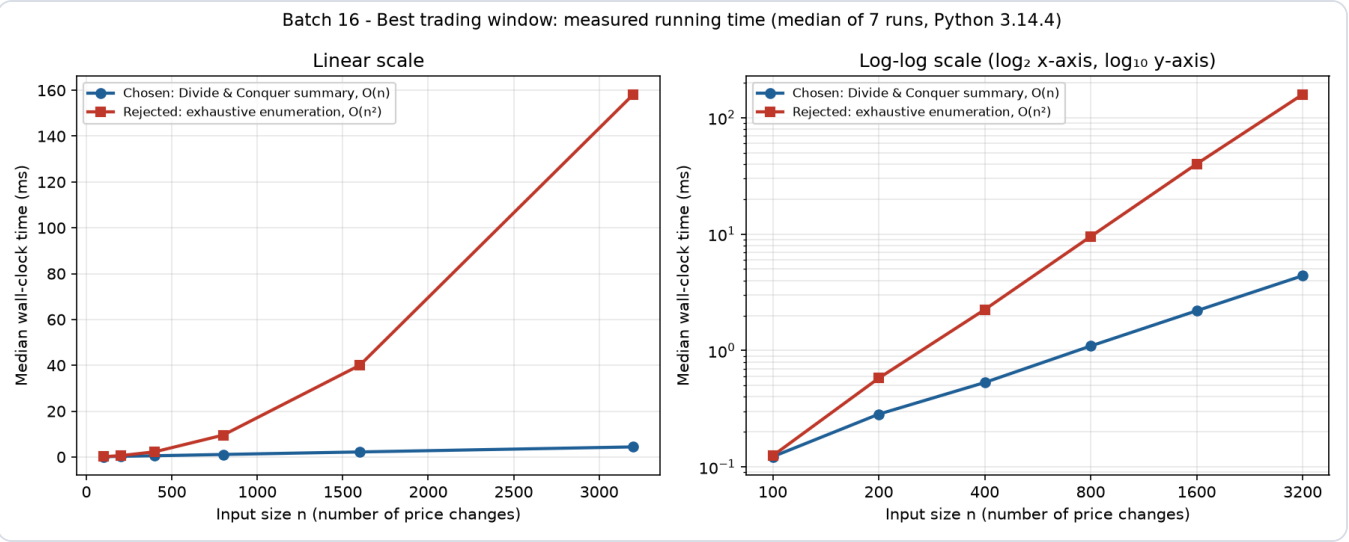
+3	−5	+8	−2	+6	−9	+4	+7	−3	−6	+5	+2
1	2	3	4	5	6	7	8	9	10	11	12

₹23 = [3,5] (12) + [7,8] (11)

That is ₹9 more than the single window, because the pair skips the −9 on day 6.

Tested against exhaustive enumeration of all windows and disjoint pairs (600 random arrays). In a distributed setting, use a parallel prefix/suffix scan or an extended associative summary.

Tests and measured performance



Measured with CPython 3.14.4 on Windows 11. Median of 7 runs after a warm-up, GC paused, same seeded input for both algorithms. Answers are asserted equal before timing.

n	D&C (ms)	Exhaustive (ms)
100	0.12	0.12
200	0.28	0.58
400	0.53	2.25
800	1.09	9.53
1,600	2.20	40.03
3,200	4.39	158.21

Doubling n: about **×2** for D&C and about **×4** for exhaustive search.

24 / 24 unit tests pass (given instance, edge cases, ties, 2,000 random brute-force checks, associativity, segment tree, sparse table, two windows).

Chosen algorithm at larger n (CPython, median of 3): n = 24,000: 0.05 s; n = 240,000: 0.35 s; n = 1,200,000: 9.29 s; n = 2,400,000: 18.90 s. This shows the pure-Python constant factor and does not represent a cluster measurement.

CONCLUSION

Takeaways and real-world reference

- Best window for the instance: **₹14, days 3–8**. With two windows: **₹23**.
- A 4-field summary with an O(1) associative merge makes the problem **composable**.
- 8 servers: Θ(n) work, 7 merges in 3 rounds, 64 integers sent, ideal time O(n/p + log p).
- For a single sequential scan, Kadane is equally Θ(n) and leaner.
- For repeated [L, R] queries with updates, use a segment tree of summaries.
- Greedy, Backtracking and Branch & Bound were rejected with evidence. DP is correct but not the primary architecture.

MapReduce (Dean & Ghemawat, OSDI 2004)

Map on local splits, then reduce: the same shape as local summaries plus a coordinator merge. **Difference:** our reduce is order-sensitive (associative, not commutative), and we only simulate the servers.

research.google/pubs/mapreduce-simplified-data-processing-on-large-clusters

cp-algorithms: Segment Tree

"Finding subsegments with the maximal sum": the same (sum, prefix, suffix, answer) node.
cp-algorithms.com/data_structures/segment_tree.html